ROS Robot Autonomous Navigation

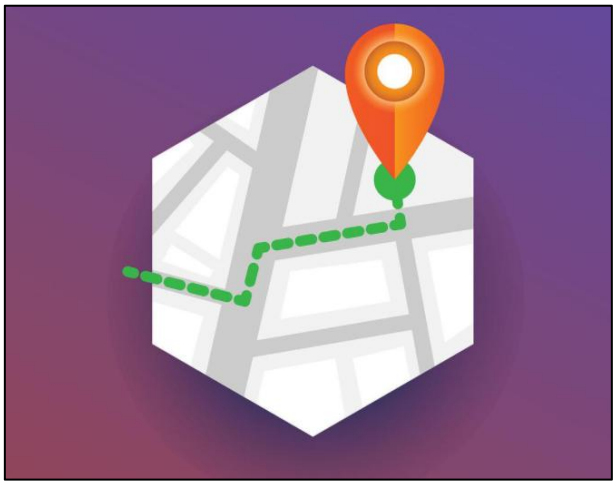
1. Description

Autonomous navigation involves guiding a device along a predefined route from one point to another. It finds application in various domains:

Land Applications: Including autonomous vehicle navigation, vehicle tracking and monitoring, intelligent vehicle information systems, Internet of Vehicles applications, railway operation monitoring, etc.

Navigation Applications: Encompassing ocean transportation, inland waterway shipping, ship berthing and docking, etc.

Aviation Applications: Such as route navigation, airport surface monitoring, precision approach, etc.



ROS (Robot Operating System) follows the principle of leveraging existing solutions and provides a comprehensive set of navigation-related function packages. These packages offer universal implementations for robot navigation, sparing developers from dealing with complex low-level tasks like navigation algorithms and hardware interactions. Managed and maintained by professional R&D personnel, these implementations allow developers to focus on higher-level functions. To utilize the navigation function, developers simply need to configure relevant parameters for their robots in the provided configuration files. Custom requirements can also be accommodated through

secondary development of existing packages, enhancing research and development efficiency and expediting product deployment.

In summary, ROS's navigation function package set offers several advantages for developers. Developed and maintained by a professional team, these packages provide stable and comprehensive functionality, allowing developers to concentrate on upper-layer functions, thus streamlining development processes.

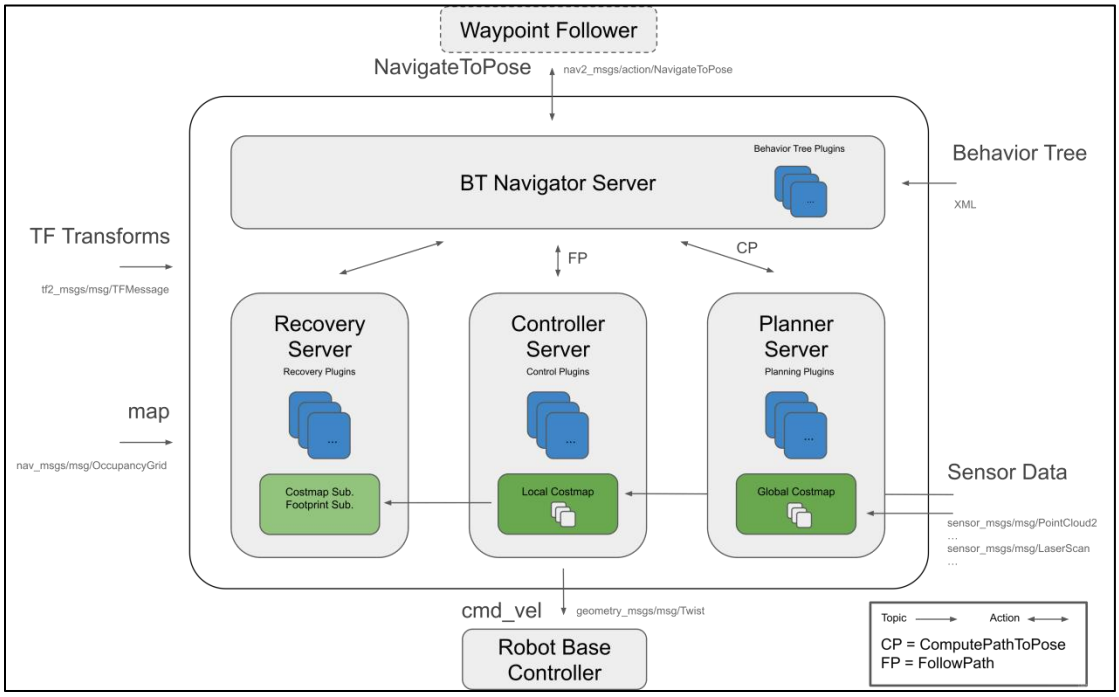
2. Package Explanation

This section will provide a detailed analysis and explanation of the navigation function package, including the usage of parameters.

2.1 Principle Structure Framework Explanation

The Nav2 project inherits and promotes the spirit of the ROS Navigation Stack. The project strives to safely navigate mobile robots from point A to point B. Nav2 can also be applied to other applications, including robot navigation tasks such as dynamic point tracking, which involves dynamic path planning, velocity calculation, obstacle avoidance, and recovery behaviors.

Nav2 utilizes behavior trees to invoke modular servers to accomplish an action. These actions can involve path computation, control forces, recovery, or any other navigation-related operation. These are achieved through independent nodes communicating with ROS Action servers and the behavior tree (BT). You can refer to the diagram below for a preliminary understanding of Nav2's architecture:



The architecture diagram above can be further explained as one major component and three minor components, totaling four services.

Major:

BT Navigator Server: This service organizes and invokes the following three minor services.

Minor:

Planner Server: Responsible for path planning, the planner server computes paths based on selected naming conventions and algorithms. These paths are essentially routes calculated on the map.

Controller Server: Also known as the local planner in ROS 1, this server provides the method for the robot to follow the globally computed path or complete local tasks. In simple terms, it controls the robot's movement based on the planned route.

Recovery Server: The backbone of the fault-tolerant system, the recovery server handles unknown or faulty conditions autonomously. Its goal is to manage unexpected situations the robot may encounter and recover autonomously. For example, if the robot falls into a hole while moving, the

recovery server finds a way to get it out.

By continuously switching between planning paths, controlling the robot along the path, and autonomously recovering from issues, the robot achieves autonomous navigation.

During robot navigation, relying solely on the original map generated by SLAM is insufficient. The robot may encounter new obstacles during movement, or it may find that obstacles in certain areas of the original map have disappeared. Therefore, the maintained map during robot navigation is dynamic. Depending on the update frequency and purpose, it can be categorized into the following two types:

Global Costmap:

The global costmap is mainly used for global path planners. As seen in the structure diagram above, it is located within the Planner Server.

It typically includes the following layers:

Static Map Layer: This layer consists of the static map usually created by SLAM.

Obstacle Map Layer: This layer is used to dynamically record obstacle information perceived by sensors.

Inflation Layer: This layer inflates (expands outward) the maps from the previous two layers to prevent the robot's shell from colliding with obstacles.

Local Costmap:

The local costmap is primarily used for the local path planner, as seen in the Controller Server in the architecture diagram above. It typically consists of the following layers:

Obstacle Map Layer: This layer records dynamically sensed obstacle information from sensors.

Inflation Layer: This layer inflates (expands outward) on top of the obstacle map layer to prevent the robot's shell from colliding with obstacles.

2.2 Install Navigation Package

Note: The navigation package is pre-installed on the robot. You do not need to install it again. This section is for learning only.

The navigation system takes navigation targets, localization information, and map information as inputs and outputs the actual control quantities that manipulate the robot. You need to know where the robot is and where the robot needs to go.

There are two ways to install the navigation package:

1) Install it directly into the system using apt-get, with the commands:

"sudo apt install ros-yoursystem-navigation2"; "yoursystem" is your ROS version. The ROS system installed on MentorPi is the Humble version.

```
sudo apt install ros-humble-navigation2
```

"sudo apt install ros-yoursystem-nav2-bringup"; "yoursystem" is your ROS version. The ROS system installed on MentorPi is the Humble version.

```
sudo apt install ros-humble-nav2-bringup
```

Note: "yoursystem" represents your ROS2 version, which can be viewed by entering "echo $ROS_DISTRO" in the terminal.

```
> echo $ROS_DISTRO
humble
```

2) Download the source code of the package and manually compile and install it. If you only need to use the navigation package for learning, install it in binary form for convenience. If you need to modify the code in the package to improve the algorithm, manually install it through the source code.

Navigation package wiki link: https://wiki.ros.org/Robots/Nav2